Breaking Up With **My Default Stack**

Patrick Favre

Lead Developer, Senacor

15+ years experience in professional software development. 10+ Projects. 4+ industries

Java Veteran

Starting out with JBoss SEAM (2009), 5y Android, Spring 3/4/Boot 1–3, Java 5-17

Current Focus

Architecture (tech/sociotech), Cloud/Backend. Industry focus FinTech/Banking currently Supervisory

github.com/patrickfav

i love the java ecosystem

... then I was put on a node project...

..., and now my reality is serverless

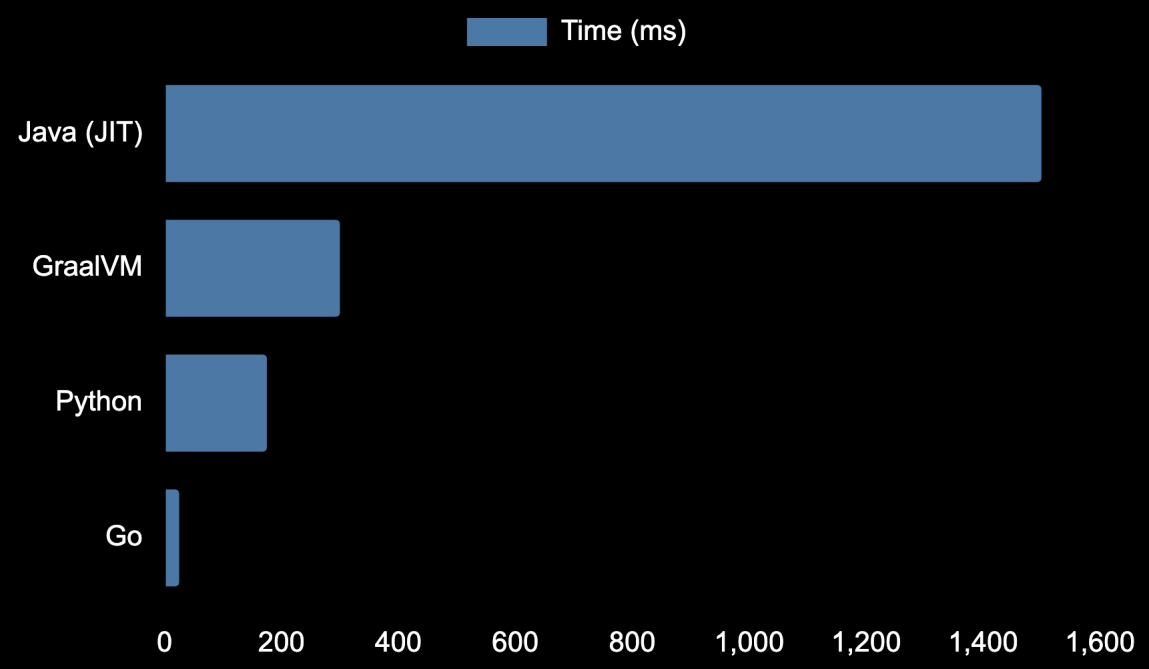
RUNTIME BEHAVIOUR

Node startup times

TYPICAL STARTUP TIME

20-200ms

Fast enough for syncronous cold start requests



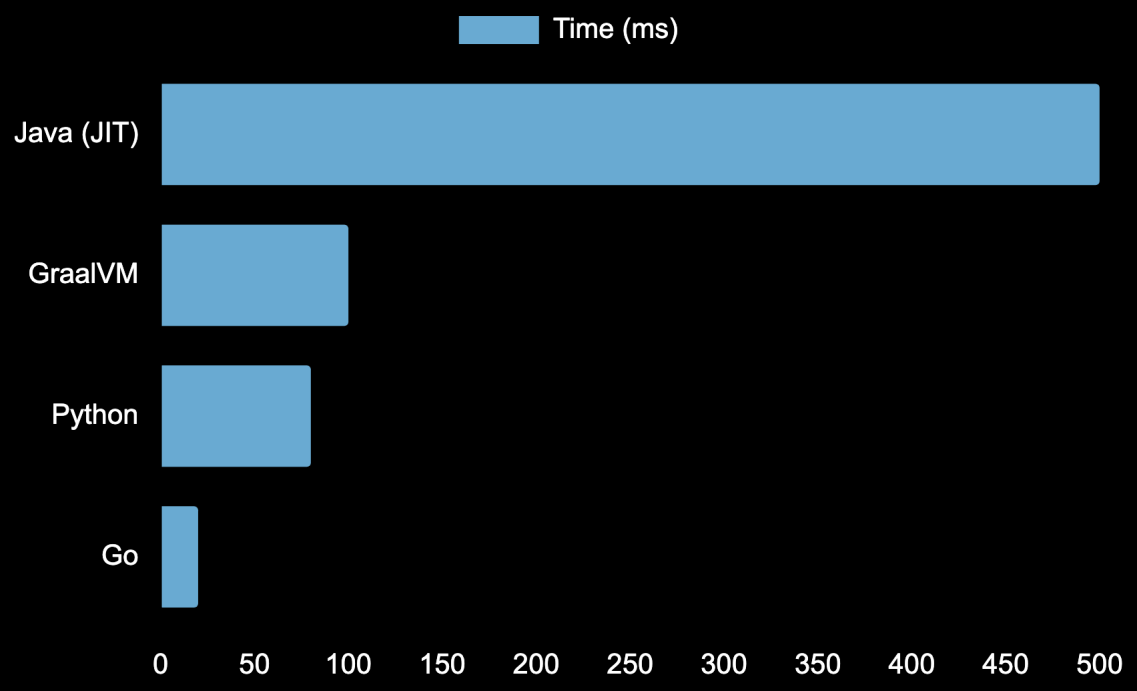
RUNTIME COMPARISON (MS)

RUNTIME BEHAVIOUR

Node RAM usage

TYPICAL RAM USAGE

30-300MB



RUNTIME COMPARISON: RAM

REQUEST MODEL

Non Blocking vs. Threaded

TRADITIONAL THREADED

One Thread per Request

Blocking I/O. Each connection consumes significant memory and waits for database/disk responses.

NODE.JS EVENT LOOP

Single-Threaded I/O

Non-blocking. Offloads I/O tasks and continues processing other requests in the event loop.

DEVELOPER EXPERIENCE

Async/Await vs. Java Reactive Models

Async Flow, Serial Feel

Node.js leverages JavaScript's **Asynchronous** nature, but modern syntax allows us to write it like traditional blocking code.

→ No complex thread management

→ Readable business logic

However, requires understanding of micro and macro task queue which might run into starvation bugs if used incorrectly.

vs.

Forces Observable APIs

In WebFlux, reactive types (Mono/Flux) are infectious. They must propagate through your **entire codebase**.

→ Controllers, Services, Repos... all reactive

→ Complex operator nesting (flatMap, zip)

→ Harder to debug and stack trace

SIDE-BY-SIDE COMPARISON

SYNTACTIC SUGAR VS. FLOWABLE COMPLEXITY

```typescript
async function saveOrder(order: Order) {
  const user = await db.users.find(order.uid);
  const result = await db.orders.insert(order);
  await email.send(user.email, "Success!");
  return result;
}

// Mock DB
const db = {
  users: {
    find: async (id: string): Promise<User | null> => {
      return { id, email: "user@example.com" };
    },
  },
  ...
};

//Service
const emailService = {
  send: async (email: string, message: string):
  Promise<void> => {
    console.log(`Email sent to ${email}: ${message}`);
  },
};
```

NODE.JS

```java
public Mono<Order> saveOrder(Order order) {
  return userRepository.findById(order.getUid())
    .flatMap(user -> orderRepository.save(order)
      .flatMap(result -> emailService.send(user.getEmail())
        .thenReturn(result)));
}

// Repository
public interface UserRepository {
    Mono<User> findById(String id);
}

public interface OrderRepository {
    Mono<Order> save(Order order);
}

// Service
public class EmailService {
    public Mono<Void> send(String email, String message) {
        return Mono.fromRunnable(() ->
            System.out.println("Email sent to " + email)
        );
    }
}
```

SPRING WEBFLUX

FOR REFERENCE: COROUTINES

```
1  suspend fun saveOrderParallel(order: Order): Order = coroutineScope {
2      val userDeferred = async { userRepository.findById(order.uid) }
3      val saveDeferred = async { orderRepository.save(order) }
4
5      val user = userDeferred.await()
6          ?: throw IllegalStateException("User not found")
7
8      val savedOrder = saveDeferred.await()
9
10     emailService.send(user.email, "Order success!")
11
12     savedOrder
13 }
```

LANGUAGE

Typescript: From the Java Developers Perspective

CONFIGURATION

Strict tsconfig

"Strict" mode is the baseline for usability. It turns the language into a reliable tool rather than a suggestion.

```
1 {
2   "strict": true,
3   "noImplicitAny": true,
4   "exactOptionalPropertyTypes": true
5 }
```

LIMITATION

Limited by JS

TS community does not like features that are "TS" only and can't just be erased to create JS. Popular example: Enums and namespaces.

LANGUAGE

Typescript allows for sophisticated infered types

TS has no type information during runtime, therefore schema validators, like Zod are often used.

TS allows to infer other, related, types from e.g. a schema.

```
1  import { z } from "zod";
2
3  export const CustomerSchema = z.object({
4    id: z.string().uuid(),
5    firstName: z.string().min(1),
6    email: z.string().email().optional(),
7    loyaltyPoints: z.number().int().nonnegative(),
8  });
9
10 /**
11  * Infer TypeScript type from schema
12  */
13 export type Customer = z.infer<typeof CustomerSchema>;
14
15 /*
16  type Customer = {
17     id: string;
18     firstName: string;
19     email?: string | undefined;
20     loyaltyPoints: number;
21  }
22 */
```

LANGUAGE

Typescript has expressive type-level language

The 20+ built-in utility types help to minimize type duplication by creating adapted infered types.

```typescript
// 1. infer return type with ReturnType<>
function createCustomer() {
  return {
    name: "Alice",
    age: 30,
  };
}

type Customer = ReturnType<typeof createCustomer>; // { name: string; age: number;}

// 2. Extracts parameter types of a function as a tuple
function updateCustomer(id: string, age: number) {
  return true;
}

type UpdateParams = Parameters<typeof updateCustomer>; // [id: string, age: number]

//3. Remove properties from a type
type User = {
  id: string;
  email: string;
  password: string;
};

type PublicCustomer = Omit<User, "password">; // { id: string, email: string;}
```

ECOSYSTEM

Nodes Ecosystem is similarly rich

BUILD & PACKAGE

Maven / Gradle

The classic Java standard for dependency management and lifecycle.

Vite + pnpm

Lightning fast HMR and efficient, symlink-based package resolution.

PACKAGING & DEPLOYMENT

JAR/WAR

Explicit compiled runtime package that might be self-contained or needs dependency resolution.

npm packages

Mainly zipped source code. Either with dependency graph or self-contained with node_moduels or packaged via esbuild/WebPack.

FRAMEWORKS

Node Web Application Frameworks are mostly "minimalistic"

PHILOSOPHY

Minimalism & Speed

Most Node frameworks prioritize a small footprint and raw performance. They provide the bare essentials, letting developers bolt on only what they need.

Express | Fastify | Hono

THE TRADE-OFF

Architectural Ownership

Being **unopinionated** means the framework won't tell you how to structure your folders, handle DI, or manage state.

REQUIREMENT

Requires **experienced developers** to define and enforce a consistent architecture from day one.

FRAMEWORKS

Introduce "Spring" for Node: NestJS

THE BRIDGE

Spring Boot for TypeScript

NestJS is heavily inspired by Angular and Spring. It brings enterprise-grade structure to Node.js, solving the "architecture-less" problem of Express.

1:1 MENTAL MODELS

Spring Bean	Nest Provider
@Component	@Injectable()
Auto-wiring	Constructor DI

PARADIGMS AND FEATURES

Modular Architecture

Explicit module definitions that mirror Spring's package-based organization.

Decorators

Uses TS Decorators just like Java Annotations for Routing, DI, and Guards.

Built-in Middlewares

Interceptors, Pipes, and Exception Filters work like Spring Interceptors/AOP.

Testing Framework & CLI

Similar to Spring Boot Test helps with integration tests and CLI to bootstrap projects.

```
1 @Controller('users')
2 export class UserController {
3   constructor(private userService: UserService) {}
4   @Get()
5   findAll() { return this.userService.findAll(); }
6 }
```

FRAMEWORKS

NestJS Example

Controller

```
@Controller('users')
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.userService.findById(id);
  }

  @Post()
  create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }
}
```

Service

```
@Injectable()
export class UserService {
  constructor(private readonly repo: UserRepository) {}

  async findById(id: string) {
    const user = await this.repo.findById(id);

    if (!user) {
      throw new NotFoundException(`User ${id} not found`);
    }

    return user;
  }

  create(dto: { name: string; email: string }) {
    return this.repo.save(dto);
  }
}
```

Model

```
import { IsEmail, IsString } from 'class-validator';

export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;
}
```

Module

```
import { Module } from '@nestjs/common';

@Module({
  controllers: [UserController],
  providers: [UserService, UserRepository],
})
export class UsersModule {}
```

SECURITY

The ugly: NPMs constant threat of supply chain attack

THE PROBLEM

Dependency Bloat

Modern JS projects often inherit 1000+ transitive dependencies. Every single one is a potential vector for malicious code injection.

MITIGATION TECHNIQUES

Release Age Filter

Using pnpm to enforce a minimum package age to avoid "day zero" exploits.

Script Whitelisting

Disabling arbitrary execution by only allowing trusted postinstall hooks.

Advanced Auth

Leveraging GitHub's improved 2FA and WebAuthn for secure account access.

Trust Policies

Implementing pnpm trustPolicy and ignoring exotic/unverified dependencies.

ECOSYSTEM

Summary & Takeaway

Performance & Efficiency

Node starts reasonably fast and maintains an efficient RAM footprint compared to the JVM.

Non-Blocking I/O

At its core, Node is a non-blocking webserver, perfectly suited for high-concurrency workloads.

The Language

TypeScript is fairly easy to learn for Java developers, offering the type safety and patterns you would expect.

Mature Ecosystem

The ecosystem has matured significantly, providing robust library support for enterprise requirements. Still fragmented and a bit chaotic.

Opinionated Options

Frameworks like NestJS provide "opinionated" options that feel familiar and bridge the architectural gap.

The Security Reality

Critical issues with Supply Chain Security remain the primary hurdle and require active management.

QUESTIONS?

:)